

Corso Python

Modulo 6: Data Handling & Automation



Collegamento al Modulo Precedente

Nel **Modulo 5** abbiamo imparato:

- ☒ Virtual environments e pip
- ☒ File I/O (TXT, CSV, JSON)
- ☒ Introduzione a Pandas (read_csv, filtri base)
- ☒ API con requests

Ora andiamo in profondità:

- **NumPy:** Il motore dei calcoli numerici
- **Pandas avanzato:** Pulizia, aggregazioni, merge
- **Automazione:** Pipeline di elaborazione dati

Agenda del Modulo (4 Ore)

Questo è il cuore dell'automazione aziendale. Impareremo a manipolare i dati come professionisti.

1. **L'Ecosistema Data:** NumPy e Pandas sotto il cofano.
2. **Il DataFrame:** Anatomia di una tabella in memoria.
3. **Manipolazione:** Filtrare, selezionare e trasformare.
4. **Pulizia Dati:** Gestire i `NaN` e correggere i tipi.
5. **Analisi & GroupBy:** Aggregare i dati per ottenere insight.
6. **Automazione:** Esportare report pronti per l'uso.

1. L'Ecosistema Data

NumPy e Pandas

Perché Servono Librerie Specializzate?

Le liste Python sono **versatili ma lente** per calcoli numerici.

Operazione	Liste Python	NumPy
Somma 1M elementi	~100ms	~1ms
Moltiplicazione	Loop manuale	Vettoriale
Memoria	Oggetti separati	Array contiguo
Tipi misti	Sì	No (ottimizzato)

L'ecosistema Data Science in Python

Frameworks:

NumPy (calcoli numerici)



Pandas (tabelle e dati)



Matplotlib/Seaborn (visualizzazione)



Scikit-learn (machine learning)

NumPy: Il Motore Sotto il Cofano

Prima di Pandas, c'è **NumPy** (Numerical Python). È il fondamento di tutto l'ecosistema scientifico Python.

Perché NumPy è così veloce?

- **Array contigui in memoria:** I dati sono memorizzati uno accanto all'altro (non sparsi come nelle liste)
- **Operazioni in C:** Il codice critico è scritto in C, non in Python
- **Vettorizzazione:** Una singola istruzione opera su tutti gli elementi
- **Cache-friendly:** La CPU può pre-caricare i dati efficientemente

Numpy

Caratteristiche chiave:

- Array multidimensionali super efficienti (ndarray)
- Broadcasting: operazioni tra array di forme diverse
- Pandas è costruito *sopra* NumPy

```
pip install numpy
```

```
import numpy as np # Convenzione: alias 'np'
```

NumPy: Creare Array

```
import numpy as np

# Da lista Python
arr = np.array([1, 2, 3, 4, 5])

# Array di zeri o uni
zeri = np.zeros(5)           # [0. 0. 0. 0. 0.]
uni = np.ones((3, 3))        # Matrice 3x3 di 1

# Sequenze
seq = np.arange(0, 10, 2)     # [0 2 4 6 8]
lin = np.linspace(0, 1, 5)    # [0. 0.25 0.5 0.75 1. ]

# Array casuali
random = np.random.rand(3, 3) # Valori 0-1
normale = np.random.randn(1000) # Distribuzione normale
```

NumPy: Operazioni Vettoriali

La potenza di NumPy: operazioni su **tutti gli elementi** senza loop.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

# Operazioni matematiche (elemento per elemento)
print(arr * 10)      # [10 20 30 40 50]
print(arr ** 2)      # [ 1  4  9 16 25]
print(arr + 100)     # [101 102 103 104 105]

# Funzioni matematiche
print(np.sqrt(arr))  # [1.    1.41 1.73 2.    2.24]
print(np.sum(arr))   # 15
print(np.mean(arr))  # 3.0
print(np.std(arr))   # 1.41

# Confronti (restituiscono array booleani)
print(arr > 3)       # [False False False  True  True]
```

NumPy: Array Multidimensionali

```
import numpy as np

# Matrice 2D (3 righe x 4 colonne)
matrice = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])

print(matrice.shape)      # (3, 4)
print(matrice.ndim)      # 2 (dimensioni)
print(matrice.size)      # 12 (elementi totali)

# Slicing
print(matrice[0, :])      # Prima riga: [1 2 3 4]
print(matrice[:, 0])      # Prima colonna: [1 5 9]
print(matrice[0:2, 1:3])  # Sotto-matrice

# Reshape
piatto = matrice.reshape(12)      # Array 1D
altro = matrice.reshape(2, 6)     # 2x6
```



Esercizi Intermedi: NumPy

Pratica Prima di Proseguire

Esercizio 1.1: Array e Operazioni Vettoriali

Obiettivo: Creare array NumPy e confrontare performance con liste Python.

```
import numpy as np
import time

# Task:
# 1. Crea un array NumPy con i numeri da 0 a 999999
# 2. Calcola il quadrato di ogni elemento
# 3. Confronta il tempo con una lista Python equivalente
```

Esercizio 1.2: Array Multidimensionali

Obiettivo: Lavorare con matrici e operazioni su righe/colonne.

```
import numpy as np
```

```
# Task:
```

```
# 1. Crea una matrice 4x3 con numeri da 1 a 12
```

```
# 2. Accedi all'elemento in posizione (2, 1)
```

```
# 3. Estrai la seconda riga completa
```

```
# 4. Calcola la somma di ogni colonna
```

```
# 5. Trasforma la matrice in un array 1D di 12 elementi
```

Pandas: Excel Programmabile

Se NumPy gestisce i numeri puri, **Pandas** gestisce le tabelle con dati eterogenei: testo, numeri, date, categorie.

Quando usare Pandas invece di Excel?

- File > 100.000 righe (Excel rallenta)
- Operazioni ripetitive (automatizzabili con script)
- Dati da fonti multiple (API, database, CSV)
- Necessità di riproducibilità (stesso script = stesso risultato)
- Integrazione con altri sistemi (report automatici, ML)

Usare Pandas in Python

Il nome "**Pandas**" deriva da "Panel Data", un termine econometrico per dataset multidimensionali.

```
pip install pandas
```

```
import pandas as pd  # Convenzione: alias 'pd'
```

2. Il DataFrame

La struttura dati regina

Le Due Strutture di Pandas

Pandas offre due strutture dati fondamentali:

Struttura	Dimensioni	Analogia	Quando usarla
Series	1D	Una colonna Excel	Singola variabile, risultato di selezione
DataFrame	2D	Un foglio Excel completo	Dati tabulari completi

Series: È un array con etichette (indice). Ogni colonna di un DataFrame è una Series.

DataFrame: È una collezione di Series allineate sullo stesso indice. Puoi pensarlo come un dizionario di Series.

Series e DataFrame

```
import pandas as pd

# Series: array 1D con indice
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s["b"]) # 20

# DataFrame: tabella 2D
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
```

Relazione: `DataFrame = {"col1": Series1, "col2": Series2, ...}`

Anatomia di un DataFrame

Il DataFrame è una tabella con etichette, composta da:

	Colonne (Columns)				
	↓	↓	↓		
	Nome	Età	Città		
Index	0	Anna	25	Roma	← Righe
(Indice)	1	Luca	30	Milano	
	2	Sara	28	Napoli	

- **Data:** I valori veri e propri
- **Index:** Le etichette delle righe (default: 0, 1, 2...)
- **Columns:** Le intestazioni delle colonne
- **dtypes:** I tipi di dato di ogni colonna

Creare un DataFrame

```
import pandas as pd

# 1. Da dizionario (più comune)
data = {
    "Nome": ["Anna", "Luca", "Sara"],
    "Età": [25, 30, 28],
    "Città": ["Roma", "Milano", "Napoli"]
}
df = pd.DataFrame(data)

# 2. Da lista di dizionari (es. da API JSON)
records = [
    {"Nome": "Anna", "Età": 25},
    {"Nome": "Luca", "Età": 30}
]
df = pd.DataFrame(records)

# 3. Da array NumPy
import numpy as np
df = pd.DataFrame(np.random.rand(3, 4), columns=["A", "B", "C", "D"])
```

Leggere Dati da File

Pandas legge praticamente qualsiasi formato:

```
import pandas as pd

# CSV (il più comune)
df = pd.read_csv("dati.csv")
df = pd.read_csv("dati.csv", sep=";", encoding="utf-8")

# Excel
df = pd.read_excel("report.xlsx", sheet_name="Vendite")

# JSON
df = pd.read_json("api_response.json")

# Da URL direttamente!
url = "https://example.com/dataset.csv"
df = pd.read_csv(url)

# SQL (richiede SQLAlchemy)
from sqlalchemy import create_engine
engine = create_engine("sqlite:///database.db")
df = pd.read_sql("SELECT * FROM vendite", engine)
```

Esplorare il DataFrame

Prima di analizzare, **esplora** sempre i tuoi dati:

```
# Dimensioni e struttura
print(df.shape)      # (righe, colonne)
print(df.columns)    # Nomi colonne
print(df.dtypes)     # Tipi di dato
print(df.info())     # Riepilogo completo

# Visualizzare i dati
print(df.head())      # Prime 5 righe
print(df.head(10))    # Prime 10 righe
print(df.tail())      # Ultime 5 righe
print(df.sample(5))   # 5 righe casuali

# Statistiche descrittive
print(df.describe())  # Solo numeriche
print(df.describe(include="all")) # Tutte le colonne
```

Attributi Utili del DataFrame

```
# Informazioni di base
print(df.shape)      # (100, 5) - 100 righe, 5 colonne
print(len(df))       # 100 - numero righe
print(df.columns)    # Index(['Nome', 'Età', ...])
print(df.index)      # RangeIndex(start=0, stop=100)

# Tipi di dato
print(df.dtypes)
# Nome      object   (stringa)
# Età       int64    (intero)
# Prezzo    float64  (decimale)
# Data      datetime64[ns]

# Valori unici
print(df["Città"].unique())      # Array valori unici
print(df["Città"].nunique())     # Conteggio valori unici
print(df["Città"].value_counts()) # Frequenza per valore
```



Esercizi Intermedi: DataFrame Basics

Pratica Prima di Proseguire

Esercizio 2.1: Creare DataFrame da Dati

Obiettivo: Creare DataFrame in diversi modi ed esplorarne la struttura.

```
import pandas as pd

# Task:
# 1. Crea un DataFrame da questo dizionario:
#    {"Prodotto": ["Laptop", "Mouse", "Tastiera"],
#     "Prezzo": [999, 25, 75],
#     "Quantità": [5, 50, 20]}
# 2. Visualizza le prime righe con .head()
# 3. Stampa le dimensioni con .shape
# 4. Mostra i tipi di dato con .dtypes
# 5. Ottieni statistiche descrittive con .describe()
```

Esercizio 2.2: Leggere e Esplorare CSV

Obiettivo: Caricare un CSV e analizzarne la struttura.

```
import pandas as pd

# Prima, crea un CSV di esempio
dati = {
    "Nome": ["Alice", "Bob", "Charlie", "Diana"],
    "Età": [25, 30, 35, 28],
    "Città": ["Roma", "Milano", "Roma", "Napoli"],
    "Stipendio": [30000, 45000, 50000, 35000]
}
pd.DataFrame(dati).to_csv("persone.csv", index=False)

# Task:
# 1. Carica il file persone.csv
# 2. Visualizza shape, columns e info()
# 3. Mostra quante persone ci sono per città (.value_counts())
# 4. Trova lo stipendio medio con .mean()
```

Esercizio 2.3: Selezione Base

Obiettivo: Accedere a righe e colonne in modi diversi.

```
import pandas as pd
```

```
df = pd.DataFrame({  
    "Nome": ["Mario", "Luigi", "Peach"],  
    "Punti": [100, 85, 92],  
    "Livello": [5, 4, 5]  
})
```

Task:

- # 1. Seleziona solo la colonna "Nome"*
- # 2. Seleziona le colonne "Nome" e "Punti"*
- # 3. Usa .loc per selezionare la prima riga*
- # 4. Usa .iloc per selezionare le prime 2 righe*
- # 5. Seleziona il valore "Punti" di "Luigi" usando .loc*

3. Manipolazione Dati

Selezione, Filtraggio e Trasformazione

Selezionare Colonne

```
# Singola colonna → restituisce una Series
nomi = df["Nome"]
print(type(nomi))  # pandas.core.series.Series

# Singola colonna (sintassi alternativa)
nomi = df.Nome  # Funziona se il nome non ha spazi

# Più colonne → restituisce un DataFrame
subset = df[["Nome", "Fatturato"]]
print(type(subset))  # pandas.core.frame.DataFrame

# Selezionare tutte tranne alcune
df_ridotto = df.drop(columns=["Colonna_Inutile", "Altra"])
```

Selezionare Righe: `iloc` vs `loc`

Due metodi fondamentali per selezionare righe. La distinzione è **cruciale**:

Metodo	Usa	Come funziona	Slicing
<code>iloc</code>	Posizione (numeri)	Come le liste Python	<code>[0:5]</code> esclude 5
<code>loc</code>	Etichetta (indice)	Cerca l'etichetta	<code>[0:5]</code> include 5

Quando usare quale?

- `iloc`: Quando sai la **posizione** ("voglio le prime 10 righe")
- `loc`: Quando sai l'**etichetta** ("voglio la riga con ID=42")

Attenzione: Se l'indice è numerico (0, 1, 2...), `iloc[0]` e `loc[0]` sembrano uguali, ma il comportamento dello slicing è diverso!

Esempio loc e iloc

```
# iloc: selezione per POSIZIONE (Integer Location)
df.iloc[0]           # Prima riga
df.iloc[0:5]         # Prime 5 righe (0,1,2,3,4 - esclude 5)
df.iloc[-1]          # Ultima riga
df.iloc[0, 2]         # Riga 0, colonna 2
df.iloc[:, 0:3]       # Tutte le righe, prime 3 colonne

# loc: selezione per ETICHETTA (Label)
df.loc[0]             # Riga con indice 0
df.loc[0:5]           # Righe con indice da 0 a 5 (INCLUDE 5!)
df.loc[0, "Nome"]      # Valore specifico
df.loc[:, ["Nome", "Età"]] # Tutte le righe, colonne specifiche
```

Filtraggio (Boolean Masking)

La tecnica più potente di Pandas: filtrare righe basandosi su **condizioni booleane**.

Come funziona il Boolean Masking?

1. Crei un confronto → ottieni una Series di `True/False`
2. Passi questa "maschera" al DataFrame
3. Pandas restituisce solo le righe dove c'è `True`

Perché è potente?

- Niente loop espliciti (veloce su milioni di righe)
- Leggibile: il codice "dice" cosa vuoi
- Componibile: puoi combinare più condizioni

Boolean Masking

```
# 1. Creo una "maschera" booleana
filtro = df["Importo"] > 100
print(filtro)  # Series di True/False per ogni riga

# 2. Applico la maschera al DataFrame
vendite_alte = df[filtro]  # Solo righe dove filtro è True

# Oppure in una riga (più comune)
df_milano = df[df["Città"] == "Milano"]

# Diversi operatori
df[df["Età"] ≥ 18]          # Maggiori o uguali
df[df["Nome"] ≠ "Mario"]    # Diverso da
df[df["Città"].isin(["Roma", "Milano"])]  # Valori in lista
```

Filtri Multipli e Stringhe

```
# Condizioni multiple (PARENTESI OBBLIGATORIE!)
# AND: &
target = df[(df["Età"] > 18) & (df["Città"] = "Roma")]

# OR: |
filtro = df[(df["Città"] = "Roma") | (df["Città"] = "Milano")]

# NOT: ~
non_roma = df[~(df["Città"] = "Roma")]

# Filtri su stringhe
df[df["Nome"].str.contains("Mar")]      # Contiene "Mar"
df[df["Nome"].str.startswith("A")]      # Inizia con "A"
df[df["Email"].str.endswith("@gmail.com")] # Finisce con
df[df["Nome"].str.len() > 5]            # Lunghezza > 5
```

Creare Nuove Colonne

Le operazioni sono **vettoriali**: agiscono su tutta la colonna senza loop.

```
# Colonna con valore fisso
```

```
df["Stato"] = "Attivo"
```

```
df["Anno"] = 2024
```

```
# Colonna calcolata matematicamente
```

```
df["Totale_Ivato"] = df["Imponibile"] * 1.22
```

```
df["Margine"] = df["Vendite"] - df["Costi"]
```

```
df["Margine_%"] = (df["Margine"] / df["Vendite"]) * 100
```

```
# Unire colonne di testo
```

```
df["Nome_Completo"] = df["Nome"] + " " + df["Cognome"]
```

```
# Colonna condizionale
```

```
df["Categoria"] = df["Età"].apply(lambda x: "Senior" if x > 50 else "Junior")
```

Apply: Funzioni Personalizzate

Quando serve logica complessa, usa `apply()` :

```
# Funzione personalizzata
def classifica_vendita(importo):
    if importo > 1000:
        return "Alta"
    elif importo > 100:
        return "Media"
    else:
        return "Bassa"

df["Classe"] = df["Importo"].apply(classifica_vendita)

# Apply su più colonne (axis=1 = per riga)
def calcola_totale(row):
    return row["Quantità"] * row["Prezzo"] * (1 - row["Sconto"])

df["Totale"] = df.apply(calcola_totale, axis=1)

# Lambda per casi semplici
df["Nome_Upper"] = df["Nome"].apply(lambda x: x.upper())
```

Modificare Colonne Esistenti

```
# Rinominare colonne
df = df.rename(columns={"old_name": "new_name", "Età": "Age"})

# Rinominare tutte le colonne
df.columns = ["Col1", "Col2", "Col3"]

# Riordinare colonne
df = df[["Nome", "Cognome", "Età", "Città"]] # Ordine desiderato

# Eliminare colonne
df = df.drop(columns=["Colonna_Inutile"])
df = df.drop(columns=["A", "B", "C"]) # Multiple

# Eliminare righe per indice
df = df.drop(index=[0, 5, 10])
```



Esercizi Intermedi: Manipolazione Dati

Pratica Prima di Proseguire

Esercizio 3.1: Filtraggio Condizionale

Obiettivo: Applicare filtri multipli e complessi ai DataFrame.

```
import pandas as pd

df = pd.DataFrame({
    "Prodotto": ["Laptop", "Mouse", "Tastiera", "Monitor", "Webcam"],
    "Prezzo": [999, 25, 75, 299, 89],
    "Quantità": [5, 50, 20, 10, 15],
    "Categoria": ["Elettronica", "Accessori", "Accessori", "Elettronica", "Accessori"]
})

# Task:
# 1. Filtra prodotti con prezzo > 50 E quantità < 20
# 2. Filtra prodotti Elettronica 0 con prezzo < 100
# 3. Usa .isin() per trovare solo Laptop e Monitor
# 4. Filtra prodotti il cui nome contiene "ta" (usa .str.contains())
```

Esercizio 3.2: Creare Colonne Calcolate

Obiettivo: Aggiungere nuove colonne derivate da calcoli.

```
import pandas as pd

df = pd.DataFrame({
    "Prodotto": ["Laptop", "Mouse", "Tastiera"],
    "Prezzo": [999, 25, 75],
    "Quantità": [5, 50, 20],
    "Sconto_%": [10, 0, 5]
})

# Task:
# 1. Aggiungi colonna "Valore_Stock" (Prezzo * Quantità)
# 2. Aggiungi colonna "Prezzo_Scontato" (Prezzo * (1 - Sconto_%/100))
# 3. Usa .apply() per creare colonna "Categoria_Prezzo":
#     "Economico" se < 50, "Medio" se < 500, "Premium" altrimenti
# 4. Crea colonna "Nome_Upper" con nome prodotto in maiuscolo
```

Esercizio 3.3: Boolean Masking

Avanzato

Obiettivo: Combinare condizioni complesse per filtrare dati.

```
import pandas as pd

df = pd.DataFrame({
    "Nome": ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "Età": [25, 30, 35, 28, 42],
    "Città": ["Roma", "Milano", "Roma", "Napoli", "Milano"],
    "Stipendio": [30000, 45000, 50000, 35000, 60000]
})

# Task:
# 1. Trova persone di Roma con età > 26
# 2. Trova persone con stipendio tra 35000 e 50000 (inclusi)
# 3. Trova persone di Milano 0 con stipendio > 50000
# 4. Usa .query() per trovare: "Età > 30 and Città = 'Milano'"
```

4. Pulizia Dati (Data Cleaning)

Gestire il caos del mondo reale

Perché la Pulizia è Fondamentale?

I dati reali sono **sempre sporchi**. La pulizia può richiedere fino all'**80% del tempo** di un progetto data.

Problemi comuni nei dati aziendali:

- **Valori mancanti:** Celle vuote, "N/A", "null", "-"
- **Duplicati:** Stessa riga inserita più volte
- **Tipi sbagliati:** "1.000,50" è una stringa, non un numero!
- **Formati inconsistenti:** "Roma", "roma", "ROMA", " Roma "
- **Outlier:** Età = 999, Prezzo = -50
- **Errori di encoding:** Caratteri speciali corrotti (è → ?)

Regola d'oro: *Garbage In, Garbage Out*

Se i dati in input sono sporchi, qualsiasi analisi, report o modello ML sarà **inaffidabile**.

Analisi dati step by step

Workflow consigliato:

1. **Esplora:** `df.info()`, `df.describe()`, `df.head()`
2. **Identifica:** Conta NaN, duplicati, tipi sbagliati
3. **Decidi:** Eliminare, imputare, o correggere?
4. **Pulisci:** Applica le trasformazioni
5. **Verifica:** Controlla che i problemi siano risolti

Gestire i NaN (Valori Mancanti)

`NaN` (Not a Number) indica un dato assente. Pandas lo gestisce nativamente con il tipo speciale `np.nan`.

Perché i NaN sono problematici?

- Le operazioni matematiche con NaN danno NaN (`5 + NaN = NaN`)
- Molti algoritmi ML non accettano NaN
- Possono distorcere statistiche (medie, conteggi)

Gestire i NaN

Come decidere cosa fare?

Situazione	Strategia consigliata
Pochi NaN (< 5%)	Elimina le righe
NaN in colonna target	Elimina (non puoi imputare ciò che vuoi predire)
NaN in feature numerica	Imputa con media/mediana
NaN in feature categorica	Imputa con moda o "Sconosciuto"
Molti NaN (> 30%)	Considera eliminare la colonna

```
# Identificare i NaN
print(df.isna().sum())           # Conta NaN per colonna
print(df.isna().sum().sum())     # Totale NaN nel DataFrame
print(df[df["Prezzo"].isna()])  # Righe con Prezzo mancante
```

```
# Percentuale di valori mancanti
print((df.isna().sum() / len(df)) * 100)
```

Strategie per i Valori Mancanti

```
# STRATEGIA 1: Eliminare le righe  
df_pulito = df.dropna() # Elimina righe con qualsiasi NaN  
df_pulito = df.dropna(subset=["Prezzo"]) # Solo se NaN in Prezzo  
df_pulito = df.dropna(thresh=3) # Mantieni se almeno 3 valori non-null  
  
# STRATEGIA 2: Riempire con valore fisso  
df["Prezzo"] = df["Prezzo"].fillna(0)  
df["Nome"] = df["Nome"].fillna("Sconosciuto")  
df["Attivo"] = df["Attivo"].fillna(False)  
  
# STRATEGIA 3: Riempire con statistiche  
df["Età"] = df["Età"].fillna(df["Età"].mean()) # Media  
df["Età"] = df["Età"].fillna(df["Età"].median()) # Mediana  
df["Città"] = df["Città"].fillna(df["Città"].mode()[0]) # Moda  
  
# STRATEGIA 4: Interpolazione (per serie temporali)  
df["Valore"] = df["Valore"].interpolate()
```

Gestire i Duplicati

```
# Identificare duplicati
print(df.duplicated().sum()) # Conta righe duplicate

# Vedere le righe duplicate
duplicati = df[df.duplicated(keep=False)] # Tutte le copie

# Duplicati su colonne specifiche
df[df.duplicated(subset=["Email"], keep=False)]

# Rimuovere duplicati
df_unico = df.drop_duplicates() # Righe identiche
df_unico = df.drop_duplicates(subset=["Email"]) # Per colonna
df_unico = df.drop_duplicates(subset=["Email"], keep="last") # Tieni ultima
```

Conversione dei Tipi (Casting)

Spesso i numeri arrivano come testo (es. da CSV o Excel).

```
# Verificare i tipi attuali
print(df.dtypes)

# Convertire tipi base
df["Quantità"] = df["Quantità"].astype(int)
df["Prezzo"] = df["Prezzo"].astype(float)
df["Codice"] = df["Codice"].astype(str)

# Conversione con gestione errori
df["Valore"] = pd.to_numeric(df["Valore"], errors="coerce") # Errori → NaN

# Categorie (risparmia memoria per colonne ripetitive)
df["Stato"] = df["Stato"].astype("category")
```

Lavorare con le Date

Le date sono fondamentali per l'analisi temporale.

```
# Convertire stringhe in datetime
df["Data"] = pd.to_datetime(df["Data"])
df["Data"] = pd.to_datetime(df["Data"], format="%d/%m/%Y") # Formato specifico

# Estrarre componenti dalla data
df["Anno"] = df["Data"].dt.year
df["Mese"] = df["Data"].dt.month
df["Giorno"] = df["Data"].dt.day
df["Giorno_Settimana"] = df["Data"].dt.day_name()
df["Trimestre"] = df["Data"].dt.quarter

# Filtrare per periodo
df_2024 = df[df["Data"].dt.year == 2024]
df_q1 = df[df["Data"].dt.quarter == 1]
```

Pulizia Stringhe

I dati testuali sono spesso inconsistenti.

```
# Rimuovere spazi extra
df["Nome"] = df["Nome"].str.strip()           # Spazi inizio/fine
df["Nome"] = df["Nome"].str.replace("  ", " ") # Doppi spazi

# Standardizzare case
df["Città"] = df["Città"].str.lower()         # minuscolo
df["Città"] = df["Città"].str.upper()         # MAIUSCOLO
df["Città"] = df["Città"].str.title()         # Prima Lettera Maiuscola

# Sostituire valori
df["Prezzo"] = df["Prezzo"].str.replace("€", "").str.replace(",", ".")
df["Prezzo"] = df["Prezzo"].astype(float)

# Replace con dizionario (mappatura)
df["Stato"] = df["Stato"].replace({
    "attivo": "Attivo",
    "ATTIVO": "Attivo",
    "inattivo": "Inattivo"
})
```



Esercizi Intermedi: Pulizia Dati

Pratica Prima di Proseguire

Esercizio 4.1: Gestire Valori Mancanti

Obiettivo: Identificare e gestire NaN in modi diversi.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Nome": ["Alice", "Bob", np.nan, "Diana"],
    "Età": [25, np.nan, 35, 28],
    "Email": ["alice@email.it", "bob@email.it", "charlie@email.it", np.nan],
    "Stipendio": [30000, 45000, np.nan, 35000]
})
```

Task:

- # 1. Conta quanti NaN ci sono per ogni colonna (.isnull().sum())*
- # 2. Rimuovi righe con NaN in "Nome" (dropna(subset=["Nome"]))*
- # 3. Sostituisci NaN in "Età" con la media delle età*
- # 4. Sostituisci NaN in "Email" con "non_disponibile@email.it"*
- # 5. Rimuovi la colonna "Stipendio" se ha più di 1 NaN*

Esercizio 4.2: Pulizia Stringhe

Obiettivo: Normalizzare e pulire dati testuali.

```
import pandas as pd

df = pd.DataFrame({
    "Nome": ["  alice  ", "BOB", "Charlie ", " diana"],
    "Email": ["ALICE@EMAIL.IT", "bob@email.it  ", "Charlie@Email.It", "diana@email.it"],
    "Prezzo": ["€ 25,50", "€ 30,00", "€ 15,75", "€ 42,25"]
})

# Task:
# 1. Rimuovi spazi extra da "Nome" (.str.strip())
# 2. Converti "Nome" in formato Title Case (.str.title())
# 3. Converti "Email" tutto in minuscolo e rimuovi spazi
# 4. Pulisci "Prezzo": rimuovi "€" e spazi, converti "," in "."
# 5. Converti la colonna "Prezzo" in float
```

Esercizio 4.3: Conversione Tipi e Duplicati

Obiettivo: Convertire tipi di dato e rimuovere duplicati.

```
import pandas as pd

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4],
    "Età": ["25", "30", "30", "35", "abc"], # Errore in abc
    "Data": ["2024-01-15", "2024-02-20", "2024-02-20", "2024-03-10", "2024-04-05"]
})
```

Task:

- # 1. Identifica righe duplicate (.duplicated())*
- # 2. Rimuovi righe duplicate complete (.drop_duplicates())*
- # 3. Converti "Età" in int, gestendo errori con errors='coerce'*
- # 4. Converti "Data" in datetime con pd.to_datetime()*
- # 5. Conta quanti NaN sono stati creati dalla conversione di "Età"*

5. Analisi e Aggregazione

GroupBy, Merge e Statistiche

Statistiche Descrittive

```
# Statistiche su singola colonna
print(df["Vendite"].sum())      # Somma
print(df["Vendite"].mean())    # Media
print(df["Vendite"].median())  # Mediana
print(df["Vendite"].std())     # Deviazione standard
print(df["Vendite"].min())     # Minimo
print(df["Vendite"].max())     # Massimo
print(df["Vendite"].count())   # Conteggio (esclusi NaN)
```

```
# Statistiche multiple
print(df["Vendite"].describe())
# count      100.0
# mean       150.5
# std        45.2
# min        10.0
# 25%       120.0
# 50%       150.0
# 75%       180.0
# max       300.0
```

GroupBy: Le Pivot Table di Python

Il **groupby** raggruppa i dati per categoria e applica aggregazioni.

Paradigma Split-Apply-Combine: Questo è il modello mentale per capire groupby:



1. **Split:** Dividi i dati per valore di una colonna
2. **Apply:** Applica una funzione (sum, mean, count...)
3. **Combine:** Combina i risultati in un nuovo DataFrame

GroupBy Avanzato

```
# Raggruppare per più colonne
df.groupby(["Anno", "Mese"])["Vendite"].sum()

# Più aggregazioni contemporaneamente
stats = df.groupby("Reparto")["Stipendio"].agg(["mean", "min", "max", "count"])

# Aggregazioni diverse per colonne diverse
report = df.groupby("Città").agg({
    "Vendite": "sum",
    "Quantità": "mean",
    "Cliente": "nunique" # Conteggio valori unici
})

# Rinominare le colonne risultanti
report = df.groupby("Città").agg(
    Totale_Vendite=("Vendite", "sum"),
    Media_Quantità=("Quantità", "mean"),
    N_Clienti=("Cliente", "nunique")
)
```

Ordinamento

```
# Ordinare per una colonna
df_ord = df.sort_values("Vendite")
df_ord = df.sort_values("Vendite", ascending=False) # Decrescente

# Ordinare per più colonne
df_ord = df.sort_values(["Città", "Vendite"], ascending=[True, False])

# Top N valori
top_10 = df.nlargest(10, "Vendite")
bottom_5 = df.nsmallest(5, "Prezzo")

# Ordinare per indice
df_ord = df.sort_index()
```

Unire DataFrame: Merge

Come le **JOIN** in SQL, per combinare dati da tabelle diverse basandosi su colonne in comune.

Tipi di Join (stesso concetto di SQL):

Tipo	Cosa restituisce
inner	Solo righe con corrispondenza in entrambe
left	Tutte le righe da sinistra + match da destra
right	Tutte le righe da destra + match da sinistra
outer	Tutte le righe da entrambe

Merge

Quando usare Merge?

- Dati in tabelle separate (clienti + ordini)
- Arricchire dati con informazioni aggiuntive
- Normalizzazione database (evitare duplicazione)

```
# Due DataFrame
vendite = pd.DataFrame({
    "Prodotto_ID": [1, 2, 3],
    "Vendite": [100, 200, 150]
})
prodotti = pd.DataFrame({
    "ID": [1, 2, 3, 4],
    "Nome": ["Laptop", "Mouse", "Tastiera", "Monitor"]
})
# Inner Join (solo corrispondenze)
df = pd.merge(vendite, prodotti, left_on="Prodotto_ID", right_on="ID")
# Left Join (tutte le righe da sinistra)
df = pd.merge(vendite, prodotti, left_on="Prodotto_ID", right_on="ID", how="left")
# Outer Join (tutte le righe)
df = pd.merge(vendite, prodotti, left_on="Prodotto_ID", right_on="ID", how="outer")
```

Concatenare DataFrame

Unire DataFrame "impilando" righe o colonne.

```
# Dati da più mesi
gennaio = pd.read_csv("vendite_gennaio.csv")
febbraio = pd.read_csv("vendite_febbraio.csv")
marzo = pd.read_csv("vendite_marzo.csv")

# Concatenare verticalmente (aggiungere righe)
df_anno = pd.concat([gennaio, febbraio, marzo], ignore_index=True)

# Concatenare orizzontalmente (aggiungere colonne)
df_completo = pd.concat([df1, df2], axis=1)
```

Esercizi Intermedi: Aggregazione e Analisi

Pratica Prima di Proseguire

Esercizio 5.1: GroupBy Base

Obiettivo: Raggruppare dati e calcolare statistiche aggregate.

```
import pandas as pd

df = pd.DataFrame({
    "Categoria": ["Elettronica", "Accessori", "Elettronica", "Accessori", "Elettronica"],
    "Prodotto": ["Laptop", "Mouse", "Monitor", "Tastiera", "Webcam"],
    "Vendite": [1200, 50, 800, 75, 150],
    "Quantità": [5, 20, 8, 15, 10]
})
```

Task:

- # 1. Raggruppa per "Categoria" e calcola la somma delle "Vendite"*
- # 2. Raggruppa per "Categoria" e calcola la media della "Quantità"*
- # 3. Conta quanti prodotti ci sono per categoria (.groupby().size())*
- # 4. Trova le vendite massime per categoria*

Esercizio 5.2: GroupBy Avanzato

Obiettivo: Usare aggregazioni multiple con `agg()`.

```
import pandas as pd

df = pd.DataFrame({
    "Città": ["Roma", "Milano", "Roma", "Napoli", "Milano"],
    "Nome": ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "Stipendio": [30000, 45000, 35000, 28000, 50000],
    "Età": [25, 30, 35, 28, 42]
})

# Task:
# 1. Raggruppa per "Città" e calcola sum, mean, count per "Stipendio"
#    (usa .agg(["sum", "mean", "count"]))
# 2. Usa .agg() con dizionario per applicare funzioni diverse:
#    {"Stipendio": ["mean", "max"], "Età": ["min", "max"]}
# 3. Trova la città con lo stipendio medio più alto
```

Esercizio 5.3: Merge DataFrame

Obiettivo: Unire due DataFrame con merge().

```
import pandas as pd

vendite = pd.DataFrame({
    "Prodotto_ID": [1, 2, 3, 4],
    "Quantità": [10, 5, 8, 12],
    "Prezzo": [999, 25, 299, 75]
})

prodotti = pd.DataFrame({
    "ID": [1, 2, 3, 5],
    "Nome": ["Laptop", "Mouse", "Monitor", "Tastiera"],
    "Categoria": ["Elettronica", "Accessori", "Elettronica", "Accessori"]
})

# Task:
# 1. Fai inner join tra vendite e prodotti
#    (left_on="Prodotto_ID", right_on="ID")
# 2. Fai left join (mantieni tutte le vendite)
# 3. Quante righe ha l'inner join? E il left join? Perché?
# 4. Aggiungi colonna "Totale" (Quantità * Prezzo) al risultato
```

6. Esportazione e Automazione

Report pronti per il business

Esportare i Dati

```
# CSV (il più compatibile)
df.to_csv("report.csv", index=False)
df.to_csv("report.csv", index=False, sep=";", encoding="utf-8")

# Excel (richiede openpyxl)
df.to_excel("report.xlsx", sheet_name="Dati", index=False)

# Più fogli nello stesso Excel
with pd.ExcelWriter("report_completo.xlsx") as writer:
    df_vendite.to_excel(writer, sheet_name="Vendite", index=False)
    df_clienti.to_excel(writer, sheet_name="Clienti", index=False)
    df_prodotti.to_excel(writer, sheet_name="Prodotti", index=False)

# JSON
df.to_json("dati.json", orient="records", indent=4)

# SQL
df.to_sql("tabella", engine, if_exists="replace", index=False)
```

Laboratorio 6

Automazione Processo Completa

Esercizio 1: Pipeline Pulizia Vendite

Scenario:

Ricevi ogni giorno un CSV sporco `raw_data.csv` con:

- Prezzi con simbolo valuta (es. "€ 50.00")
- Date in formato americano (MM/DD/YYYY)

Obiettivo:

1. Leggere il file
2. Pulire la colonna Prezzo (rimuovere '€' e convertire in float)
3. Gestire le righe con dati mancanti
4. Convertire le date
5. Raggruppare le vendite per Categoria
6. Esportare il report in Excel

Soluzione Esercizio 1

```
import pandas as pd

# 1. Carica
df = pd.read_csv("raw_data.csv")
print(f"Righe caricate: {len(df)}")

# 2. Pulizia Prezzo
df["Prezzo"] = df["Prezzo"].str.replace("€", "").str.strip()
df["Prezzo"] = pd.to_numeric(df["Prezzo"], errors="coerce")

# 3. Gestione NaN
df = df.dropna(subset=["Prezzo", "Categoria"])
print(f"Righe dopo pulizia: {len(df)}")

# 4. Conversione date
df["Data"] = pd.to_datetime(df["Data"], format="%m/%d/%Y")

# 5. Calcolo totale e aggregazione
df["Totale"] = df["Quantità"] * df["Prezzo"]
report = df.groupby("Categoria")["Totale"].agg(["sum", "mean", "count"])
report.columns = ["Totale_Vendite", "Media_Vendita", "N_Transazioni"]

# 6. Export
report.to_excel("Report_Oggi.xlsx")
print("Report salvato!")
```

Esercizio 2: Analisi Vendite Mensili

Obiettivo: Creare un report che mostri l'andamento delle vendite per mese.

```
import pandas as pd
# Carica dati
df = pd.read_csv("vendite_annuali.csv")
df["Data"] = pd.to_datetime(df["Data"])
# Estrai mese e anno
df["Anno_Mese"] = df["Data"].dt.to_period("M")
# Aggregazione mensile
report_mensile = df.groupby("Anno_Mese").agg({
    "Importo": "sum",
    "Ordine_ID": "nunique",
    "Cliente_ID": "nunique"
}).rename(columns={
    "Importo": "Fatturato",
    "Ordine_ID": "N_Ordini",
    "Cliente_ID": "Clienti_Unici"
})
print(report_mensile)
```

Esercizio 3: Merge di Dataset

Obiettivo: Unire dati da file diversi per creare un report completo.

```
import pandas as pd
# Carica i dataset
vendite = pd.read_csv("vendite.csv")
clienti = pd.read_csv("clienti.csv")
prodotti = pd.read_csv("prodotti.csv")
# Unisci vendite con clienti
df = pd.merge(vendite, clienti, on="Cliente_ID", how="left")
# Unisci con prodotti
df = pd.merge(df, prodotti, on="Prodotto_ID", how="left")
# Calcola metriche
df["Totale"] = df["Quantità"] * df["Prezzo"]
# Report per città del cliente
report_geo = df.groupby("Città_Cliente").agg({
    "Totale": "sum",
    "Cliente_ID": "nunique"
})
print(report_geo)
```

Esercizio 4: Dashboard Automatica

```
import pandas as pd
from datetime import datetime

def genera_dashboard(df):
    """Genera statistiche per dashboard."""

    dashboard = {
        "totale_vendite": df["Totale"].sum(),
        "media_ordine": df["Totale"].mean(),
        "n_ordini": len(df),
        "n_clienti": df["Cliente_ID"].nunique(),
        "prodotto_top": df.groupby("Prodotto")["Totale"].sum().idxmax(),
        "città_top": df.groupby("Città")["Totale"].sum().idxmax(),
        "ultimo_aggiornamento": datetime.now().strftime("%Y-%m-%d %H:%M")
    }

    return pd.Series(dashboard)

# Utilizzo
df = pd.read_csv("vendite.csv")
stats = genera_dashboard(df)
print(stats)
```

Riepilogo del Modulo

Argomento	Concetti Chiave
NumPy	Array veloci, operazioni vettoriali, reshape
DataFrame	Struttura 2D, index, columns, dtypes
Selezione	<code>iloc</code> (posizione), <code>loc</code> (etichetta), boolean mask
Pulizia	<code>dropna</code> , <code>fillna</code> , <code>astype</code> , <code>to_datetime</code>
Stringhe	<code>.str.replace()</code> , <code>.str.strip()</code> , <code>.str.lower()</code>
GroupBy	Split-Apply-Combine, aggregazioni multiple
Merge	Inner, left, outer join tra DataFrame
Export	CSV, Excel, JSON, SQL

Errori Comuni: Data Handling

Errori	Problema	Soluzione
<code>SettingWithCopyWarning</code>	Modifica su copia, non originale	Usa <code>.loc[]</code> o <code>.copy()</code>
<code>iloc</code> vs <code>loc</code> confusi	Slicing inclusivo/esclusivo diverso	<code>iloc</code> = posizione, <code>loc</code> = etichetta
<code>&</code> senza parentesi	Errore di precedenza operatori	<code>(cond1) & (cond2)</code>
Colonna non trovata	Spazi nascosti nel nome	Usa <code>.columns.str.strip()</code>
<code>apply</code> lento	Loop nascosto su milioni di righe	Preferisci operazioni vettoriali

Errori Comuni: Data Handling

```
# ❌ SettingWithCopyWarning
df_filtrato = df[df["Città"] == "Roma"]
df_filtrato["Nuova"] = 1 # Warning!

# ✅ Corretto
df_filtrato = df[df["Città"] == "Roma"].copy()
df_filtrato["Nuova"] = 1 # OK

# ❌ Errore parentesi
df[df["A"] > 5 & df["B"] < 10] # ERRORE!

# ✅ Corretto
df[(df["A"] > 5) & (df["B"] < 10)] # OK
```

Prossimo Modulo: Python Backend & APIs con FastAPI!